

Name :- Muhammmed Jasad	Roll no :- S024
Class :- SYIT	Batch :- B1
Date of Assignment :- 22-09-2026	Date of Submission :- 22-09-2026

Practical 27

Code -

```

Current File ▾ ▶ ⚙ ⋮
Jasad.scala ×
1 ▶ object Jasad extends App
2   {
3     val data = List("Red", "Blue", "Green", "Red", "Blue")
4     for (value <- data)
5     {
6       val r = if (value == "Red") 1 else 0
7       val b = if (value == "Blue") 1 else 0
8       val g = if (value == "Green") 1 else 0
9       println(value + " -> Red:" + r + " Blue:" + b + " Green:" + g)
10    }
11    println("Jasad S024")
12  }

```

Output -

```

C:\Program Files\Java\jdk1.8.0_341\bin\java.exe
Red -> Red:1 Blue:0 Green:0
Blue -> Red:0 Blue:1 Green:0
Green -> Red:0 Blue:0 Green:1
Red -> Red:1 Blue:0 Green:0
Blue -> Red:0 Blue:1 Green:0
Jasad S024
Process finished with exit code 0

```

Practical 28

Code -

```
build.sbt x Jasad.scala
1 ThisBuild / scalaVersion := "2.12.18"
2 libraryDependencies += "org.scalanlp" %% "breeze" % "2.1.0"
3 lazy val root = (project in file("."))
4   .settings(
5     name := "Prac27"
6   )
```

```
import breeze.linalg._
object Prac27 extends App {

  val A = DenseVector(2.0, 3.0)
  val B = DenseVector(5.0, 7.0)

  // Euclidean distance between A and B
  val distance = norm(A - B)

  println("Euclidean Distance: " + distance)

  val C = DenseVector(2.5, 3.5)

  // Distance from C to A and B
  val d1 = norm(C - A)
  val d2 = norm(C - B)

  if (d1 < d2) {
    println("Nearest Neighbor: A")
  } else {
    println("Nearest Neighbor: B")
  }
}
```

Output -

```
"C:\Program Files\Java\jdk1.8.0_181\bin\
Euclidean Distance: 5.0
Nearest Neighbor: A

Process finished with exit code 0
```

Practical 29

Code -

```
import org.apache.spark.sql.SparkSession
import org.apache.spark.ml.feature.VectorAssembler
import org.apache.spark.ml.classification.LogisticRegression

object Practical27 extends App { new {

  val spark = SparkSession.builder()
    .appName("Classification")
    .master("local[*]")
    .getOrCreate()

  import spark.implicits._

  val data = Seq(
    (1.0, 0.0),
    (2.0, 0.0),
    (3.0, 0.0),
    (7.0, 1.0),
    (8.0, 1.0),
    (9.0, 1.0)
  ).toDF("marks", "label")

  val a = new VectorAssembler()
    .setInputCols(Array("marks"))
    .setOutputCol("features")

  val d = a.transform(data)

  val model = new LogisticRegression()
    .setLabelCol("label")
    .setFeaturesCol("features")

  val result = model.fit(d)

  val prediction = result.transform(d)

  prediction.select("marks", "label", "prediction").show()

  spark.stop()
}
```

Output -

marks	label	prediction
1.0	0.0	0.0
2.0	0.0	0.0
3.0	0.0	0.0
7.0	1.0	1.0
8.0	1.0	1.0
9.0	1.0	1.0